

# Unified Offline Library Catalog (Kolibri + Kiwix Aggregator)

**Working name:** `libraryhub` **Project type:** Standalone Python web application **Difficulty:** Intermediate (OOP, file and database I/O, a third-party C-backed library, a small web layer, tests)

---

## 1. Context (read this first)

A community in rural Haiti has an offline "library hub": a small server (a Raspberry Pi) that runs without internet and lets nearby phones and laptops connect over local Wi-Fi to read educational content. The hub runs two separate pieces of software, each with its own web interface:

- **Kolibri** is an offline learning platform. It organizes content into structured "channels" (courses, videos, exercises, documents) and has a Kreyòl interface. It is itself a Python (Django) application.
- **Kiwix** serves `.zim` files, which are compressed offline copies of whole websites. The hub's main `.zim` is the Haitian Creole Wikipedia. Kiwix's core is C++, but it ships official Python bindings (the `libzim` package) for reading `.zim` files directly.

The problem: a visitor sees two unrelated apps on two different ports and has no single place to browse everything. **This project builds the missing front door:** one Python web app that reads what is inside both engines and presents a single, searchable catalog, with each entry linking out to open in the correct underlying engine.

**Critical guarantee that makes this project safe:** the tool is strictly **read-only and additive**. It never writes to, deletes, or reconfigures Kolibri or Kiwix. It runs as its own separate process on its own port. If it crashes or is removed, the two libraries keep working exactly as before. Nothing on the hub depends on it. This is why it is a good standalone project: there is no way for it to break the production library.

## Where the data lives

- **Kolibri** stores its content under a home directory (default `~/.kolibri/`). It also exposes a local **REST API** when the server is running, which is the cleanest way to read its channels and content nodes. This project uses the API.
  - **Kiwix** content is a folder of `.zim` files (for example `/data/zim/`). Each `.zim` is one self-contained library (one whole website). The `libzim` Python package reads their metadata and content.
-

## 2. Goal

One Python web service that scans both Kolibri and Kiwix, merges everything into a single catalog, and serves a searchable landing page where each item links to the real content in its own engine.

---

## 3. Scope

### In scope

- Reading content listings from Kolibri (via its REST API) and from Kiwix (via `libzim`).
- Normalizing both into one shared data model.
- Serving a single HTML catalog page with search and a language filter.
- Linking each catalog entry to the correct engine URL so the user can open it.

### Out of scope (do not build these)

- Modifying, importing, or deleting any Kolibri or Kiwix content.
  - Hosting or serving the actual content (Kolibri and Kiwix already do that; this app only links to them).
  - User accounts, logins, or permissions.
  - Anything that writes to the Kolibri database or the `.zim` files.
- 

## 4. Functional requirements

### Core (the minimum viable product)

1. **Discover Kiwix content.** Scan a configured folder of `.zim` files. For each file, read its metadata (title, description, language, article count) using `libzim`. Produce one catalog entry per `.zim`.
2. **Discover Kolibri content.** Query the Kolibri REST API for its channels (and optionally top-level topics). Produce one catalog entry per channel.
3. **Normalize.** Convert both into a single internal `CatalogItem` type so the rest of the app does not care which engine an item came from.
4. **Aggregate.** Combine all items from both sources into one in-memory catalog.
5. **Serve a catalog page.** A single web page that lists every item, shows its source (Kolibri or Kiwix), title, description, and language, and provides a working link to open it in the right engine.
6. **Search.** A search box that filters the catalog by matching text in titles and descriptions.

7. **Filter by language.** At minimum English, Kreyòl (`(ht)`), and French (`(fr)`).
8. **Degrade gracefully.** If the `(.zim)` folder is missing or Kolibri is not running, the app still starts and shows whatever source is available, with a clear note that the other source is unavailable.

### Stretch (optional, for extra credit or if time allows)

- **Full-text search inside `(.zim)` files** using the `(libzim)` search API, merged into the same results list as Kolibri matches.
  - **A JSON API endpoint** (for example `(/api/catalog)`) returning the catalog as JSON, in addition to the HTML page.
  - **Index caching.** Save the built catalog to a local SQLite file or JSON so the app starts instantly instead of rescanning every launch, with a "refresh" command to rebuild it.
  - **Health check** showing whether Kolibri and kiwix-serve are currently reachable.
  - **A third source.** Add Calibre-Web (an ebook library, also Python) by writing one new source module. The architecture below is designed so this is the only file you need to add.
- 

## 5. Non-functional requirements and constraints

- **Language:** Python 3.9 or newer (required for the prebuilt `(libzim)` wheels).
  - **Runs on a Raspberry Pi** (ARM / aarch64). `(libzim)` publishes ARM wheels, so no compiling is needed.
  - **Fully offline at runtime.** No internet calls when serving. All dependencies installed ahead of time.
  - **Read-only.** No code path may write to Kolibri's database or to any `(.zim)` file. Open `(.zim)` files in read mode only.
  - **Separate process and port.** Default to a port the engines do not use (for example `(8090)`). Kolibri commonly runs on `(8080)` and kiwix-serve on a configurable port such as `(8000)`.
  - **Configurable.** Paths, ports, and source URLs come from a config file or environment variables, never hard-coded.
- 

## 6. Data sources and how to read them

These are the two entry points. Everything else is your own code. Treat these as starting points, not finished functions.

Kiwix, via `(libzim)`

Open each `.zim` as an `Archive` and read its metadata keys. Reading is the only mode used here.

```
python

from libzim.reader import Archive

zim = Archive("hatian-creole-wikipedia.zim")
title = zim.get_metadata("Title").decode("utf-8")
language = zim.get_metadata("Language").decode("utf-8")
article_count = zim.article_count
```

For the stretch full-text search, `libzim` also provides `Searcher` and `Query` in `libzim.search`.

## Kolibri, via its REST API

When Kolibri is running, it serves a browsable JSON API. Use `requests` to GET the channel list and read the JSON. Confirm the exact endpoint paths against your installed Kolibri version by opening the API in a browser (the routes live under `/api/content/`, for example a channel endpoint and a contentnode endpoint). The base URL and port are configurable.

```
python

import requests

resp = requests.get("http://localhost:8080/api/content/channel/")
channels = resp.json() # list of channel dicts: name, description, etc.
```

If running Kolibri during development is inconvenient, see Section 11 for how to develop against a saved JSON fixture instead.

---

## 7. Data model

Define one normalized type that both sources produce. A dataclass is the natural choice and is a good teaching point.

`CatalogItem` fields:

- `source`: which engine the item came from (`"kolibri"` or `"kiwix"`).
- `title`: display name.
- `description`: short text, may be empty.
- `language`: language code (`"en"`, `"ht"`, `"fr"`, or other).

- `kind`: rough type (`"channel"`, `"website"`, `"book"`, etc.).
- `open_url`: the full URL that opens this item in its own engine.
- `extra`: an optional dict for source-specific details (article count, channel id, file size).

The rest of the application should only ever touch `CatalogItem` objects, never raw Kolibri JSON or raw `libzim` objects. That separation is the core design idea.

---

## 8. Architecture and module layout

The key pattern is a **Source interface**: each content engine is a class that knows how to produce `CatalogItem` objects. The catalog does not care how. Adding a new engine later (Calibre-Web) means writing one new source class and nothing else.

```
libraryhub/
  __init__.py
  config.py           # paths, ports, source URLs, read from env or a config file
  models.py          # the CatalogItem dataclass
  sources/
    __init__.py
    base.py           # Source: abstract base class defining the interface
    kiwix.py          # KiwixSource: reads .zim files with libzim
    kolibri.py        # KolibriSource: reads channels via the REST API
  catalog.py         # builds and holds the merged catalog, runs search and
filters
  web.py             # the web app: landing page, search route, optional JSON API
  cli.py             # entry point: build the index and run the server
  templates/
    index.html        # the catalog page (Jinja2 template)
  static/
    style.css
tests/
  test_kiwix.py
  test_kolibri.py
  test_catalog.py
README.md
requirements.txt
```

The `Source` interface in `base.py` should define at minimum:

```
python
```

```
from abc import ABC, abstractmethod
```

```
class Source(ABC):
    name: str

    @abstractmethod
    def discover(self) -> list["CatalogItem"]:
        """Return all catalog items this source can offer."""

    def is_available(self) -> bool:
        """Return True if this source can currently be reached."""
        return True
```

`KiwixSource` and `KolibriSource` each subclass `Source` and implement `discover()`. `catalog.py` simply loops over a list of sources, calls `discover()` on each available one, and concatenates the results.

---

## 9. Desired output and deliverables

When finished, running one command should:

1. Build the merged catalog from both sources.
2. Start a web server on the configured port.

A user visiting `http://<hub-ip>:8090` should see:

- A single page listing every item from both Kolibri and Kiwix.
- A label on each item showing which engine it lives in.
- A working "Open" link per item that loads it in the correct engine.
- A search box that filters the list by title and description text.
- A language filter offering at least English, Kreyòl, and French.

Deliverables to submit:

- The full source tree above.
  - `requirements.txt`.
  - A `README.md` with install and run instructions.
  - Passing tests for each source and for the catalog merge and search.
- 

## 10. Tools and dependencies

## 10. Tools and dependencies

requirements.txt:

```
libzim      # read .zim files (Kiwix content)
requests    # call the Kolibri REST API
flask       # web server and templating (Jinja2 ships with Flask)
pytest      # tests
```

Standard library used: `sqlite3` (only if you add caching), `pathlib`, `dataclasses`, `json`, `argparse`, `abc`.

**Web framework choice:** Flask is recommended for a class project because it is the simplest path to "serve an HTML page with a search box." Two alternatives if your brother prefers:

- **FastAPI** if he wants async and an auto-generated JSON API.
  - **Streamlit** if he wants to skip writing HTML entirely and let Python generate the whole interface. This changes the `web.py` design but the sources and catalog stay identical.
- 

## 11. Developing without the Raspberry Pi

He almost certainly will not have the hub. Both sources can be developed on a laptop:

- **Kiwix side:** download one small `.zim` from the public Kiwix library to test against. Even a tiny one works; `libzim` reads any valid `.zim`. The Haitian Creole Wikipedia `.zim` is small (around 200 MB) and is the most realistic test file.
  - **Kolibri side:** either `pip install kolibri`, run it locally, and import one small channel (it runs on any OS, not just the Pi), or save one real API response to a JSON file and have `KolibriSource` read from that fixture in development. The fixture approach lets tests run with no server at all.
  - **Tests:** write a fake source that returns a fixed list of `CatalogItem` objects so `catalog.py` can be tested in isolation, with no `.zim` files and no running Kolibri.
- 

## 12. Suggested build order

1. `models.py`: the `CatalogItem` dataclass.
2. `sources/kiwix.py`: open one `.zim`, read its metadata, return one `CatalogItem`. Print it from a script to confirm.
3. `sources/kolibri.py`: read channels from the API (or a saved JSON fixture) into `CatalogItem` objects.

4. `sources/base.py`: extract the shared `Source` interface and make both classes subclass it.
5. `catalog.py`: merge all sources into one list.
6. `web.py` and `templates/index.html`: serve the merged list as a page.
7. Add the search box and language filter.
8. `tests/`: cover each source and the catalog.
9. `README.md`.

Each step produces something runnable, so progress is visible the whole way.

---

### 13. Acceptance criteria (definition of done)

- Starting the app with at least one `.zim` file present and Kolibri running shows items from both sources on one page.
  - Every "Open" link loads the correct content in the correct engine.
  - Searching narrows the list correctly.
  - The language filter works for English, Kreyòl, and French.
  - Removing the `.zim` folder or stopping Kolibri does not crash the app; it shows the remaining source and a clear notice.
  - Tests pass with `pytest`.
  - The code never opens a `.zim` file or the Kolibri data for writing.
- 

### 14. Why this is a good class project

It exercises a clean spread of skills that most Python course rubrics reward: an abstract base class with concrete implementations (the `Source` pattern), dataclasses, file and HTTP I/O, working with a real third-party C-backed library, a small web application with templating, configuration handling, graceful error handling, and unit tests with fixtures. It is also genuinely finishable: the core is small, every milestone runs, and the stretch goals give room to go further without being required.